

UCS420: Cognitive Computing

Assignment 2 — Python Data Structures (Lists, Tuples, Sets, Dictionaries)

1. Take your roll number. Extract its individual digits, and multiply each digit by 10 to form your list L (10 numbers in list now).
 - i. Print L
 - ii. Add two numbers of your choice to L — one using `append()` and one using `insert()` at a specific position. Print L after each and ***explain in a comment what changed.***
 - iii. Remove two elements from L — one using `remove()` and one using `pop()`. Print L after each.
 - iv. Sort L in ascending order using `sort()`, then descending order using `sort(reverse=True)`. Print both.
 - v. Use slicing to print the first three elements and the last three elements of L in a single line each.
 - vi. Use list comprehension to create a new list containing only the elements of L that are greater than the average of L.
2. Create a tuple named scores containing 8 marks: use the first 8 values from your list L in Q1.
 - i. Find the highest score and its index in the tuple, and the lowest score and how many times it appears.
 - ii. Reverse the tuple and return the result as a list (tuples themselves cannot be reversed in place — explain why in a one-line comment, referring to tuple immutability).
 - iii. Ask the user to input a score. Print its first occurrence index if present in the tuple, or a not present message if not.
 - iv. Attempt to change one element of the tuple directly (e.g., `scores[0] = 100`). Run it, capture the error Python raises, and explain in one line why this happens and how it differs from a list.
 - v. Unpack the tuple into a first score, a second score, and all remaining scores using the `*` operator in a single line of unpacking.
3. Set the random seed to your own roll number using `random.seed(your_roll_number)` before generating your numbers — this keeps your list random-looking but reproducible and unique to you (a different seed produces a different list, so this cannot be copied from a classmate).
 - i. Generate a list of 100 random numbers between 100 and 900 (inclusive).
 - ii. Count and print all odd numbers in the list.
 - iii. Count and print all even numbers in the list.
 - iv. Count and print all prime numbers in the list, and additionally build the actual list of prime numbers found using list comprehension.
 - v. Print the number that occurs most frequently in your list, and how many times it occurs.
4. Using the same 8 digits of your roll number from Q1 (before multiplying by 10), create set A = {digit × 7 for each digit} and set B = {digit × 9 for each digit}. Print both sets.
 - vi. Find the union of A and B — all unique values across both sets.
 - vii. Find the intersection of A and B — values common to both.
 - viii. Find A – B and B – A separately (not combined) using `difference()`, and explain in one line how this differs from `symmetric_difference()`.
 - ix. Find the symmetric difference of A and B.
 - x. Check whether A is a subset of B, and whether B is a superset of A, using `issubset()` and `issuperset()`.
 - xi. Ask the user to input a value X. Remove X from set A if present using `discard()` (not `remove()`) — explain in one line why `discard()` is safer than `remove()` when you're not sure the value exists.

5. Consider the following dictionary:

```
my_dict = {  
    "name": "<your name>",  
    "roll_no": "<your roll number>",  
    "branch": "<your branch>",  
    "age": <your age>,  
    "city": "<your home city>"  
}
```

- i. Rename the key “city” to “location” without changing its value (do not just recreate the dictionary by hand — use pop() or dictionary comprehension so the code would work for any dictionary).
- ii. Add a new key “cgpa” with your CGPA as its value.
- iii. Update the value of “age” by increasing it by 1.
- iv. Delete the “branch” key using two different methods in two separate copies of the dictionary: once with pop() and once with the del keyword. Explain in one line how pop() differs from del (hint: what does each return?).
- v. Iterate over your dictionary using .items() and print each key-value pair as “key → value”.
- vi. Before accessing a key called “email” (which doesn't exist in your dictionary), check for its existence using the in keyword and print a safe fallback message instead of letting the program crash.
- vii. Create a second dictionary friend_dict with the same 5 original keys but a friend's (fictional) details. Merge it with your own using the {**dict1, **dict2} syntax, and explain in one line which values “win” when both dictionaries share a key.
- viii. Use dictionary comprehension to create a new dictionary containing only the key-value pairs from your original dictionary where the value is a string.